

Agentic Software Engineering: Foundational Pillars and a Research Roadmap

AHMED E. HASSAN, Queen's University, Canada

HAO LI, Queen's University, Canada

DAYI LIN, Queen's University, Canada

BRAM ADAMS, Queen's University, Canada

TSE-HSUN CHEN, Concordia University, Canada

YUTARO KASHIWA, Nara Institute of Science and Technology, Japan

DONG QIU, Huawei Canada, Canada

Agentic Software Engineering (SE 3.0) represents a new era where intelligent agents are tasked not with simple code generation, but with achieving complex, goal-oriented SE objectives. To harness these new capabilities while ensuring trustworthiness, we must recognize a fundamental duality within the SE field in the Agentic SE era, comprising two symbiotic modalities: SE for Humans and SE for Agents. This duality demands a radical reimagining of the foundational pillars of SE (actors, processes, tools, and artifacts) which manifest differently across each modality. This new vision of SE requires two distinct, purpose-built workbenches (aka tools) for these two collaborative modalities: the Agent Command Environment (ACE), a command center where humans orchestrate, mentor, and oversee agent teams while managing an inbox of agent-generated events like Merge-Readiness Packs (MRPs) and Consultation Request Packs (CRPs); and the Agent Execution Environment (AEE), a digital workbench where agents not only execute tasks but can proactively invoke human expertise when facing complex trade-offs or ambiguity. This bi-directional partnership, which supports agent-initiated human callbacks and handovers, gives rise to new, structured engineering activities (i.e., processes) that redefine human-AI collaboration, elevating the practice from agentic coding to true agentic software engineering. This paper presents the Structured Agentic Software Engineering (SASE) vision, outlining several of the foundational pillars for the future of SE. The paper culminates in a research roadmap that identifies a few key challenges and opportunities while briefly discussing the resulting impact of this future on SE education. Our goal is not to offer a definitive solution, but to provide a conceptual scaffold with structured vocabulary to catalyze a community-wide dialogue, pushing the SE community to think beyond its classic, human-centric tenets toward a disciplined, scalable, and trustworthy agentic future.

CCS Concepts: • **Software and its engineering** → **Software development techniques**; **Collaboration in software development**; **Software creation and management**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Agentic Software Engineering, AI Agent, Agentic AI, Coding Agent

ACM Reference Format:

Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. 2025. Agentic Software Engineering: Foundational Pillars and a Research Roadmap. 1, 1 (June 2025), 30 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Authors' Contact Information: [Ahmed E. Hassan](#), ahmed@cs.queensu.ca, Queen's University, Kingston, ON, Canada; [Hao Li](#), hao.li@queensu.ca, Queen's University, Kingston, ON, Canada; [Dayi Lin](#), dayi.lin@queensu.ca, Queen's University, Kingston, ON, Canada; [Bram Adams](#), bram.adams@queensu.ca, Queen's University, Kingston, ON, Canada; [Tse-Hsun Chen](#), peterc@encs.concordia.ca, Concordia University, Montreal, Canada; [Yutaro Kashiwa](#), yutaro.kashiwa@is.naist.jp, Nara Institute of Science and Technology, Ikoma, Japan; [Dong Qiu](#), dong.qiu@huawei.com, Huawei Canada, Kingston, ON, Canada.

2025. Manuscript submitted to ACM

Manuscript submitted to ACM

1

1 Introduction

The emergence of powerful autonomous agents (aka AI teammates [19]) can now write, test, and submit code. They have moved Software Engineering (SE) beyond AI-Augmented development (SE 2.0) and toward Agentic Software Engineering (SE 3.0) [19]. Frontier LLMs can generate entire micro-applications from short one-off prompts, suggesting unprecedented productivity. Yet building, and more importantly shipping, complex and trustworthy software for many evolving stakeholders requires structured, iterative, and trustworthy SE practices. The SE field therefore faces a fundamental tension between the velocity of automation and the rigor required to build trustworthy software.

Autonomous coding agents (e.g., Google’s Jules, OpenAI’s Codex, Anthropic’s Claude Code, and Cognition’s Devin) are already responsible for hundreds of thousands of merged pull requests (PR) [26]. Their hyper-productivity, however, reveals a significant “speed vs. trust” gap. Recent examinations of agent-generated code and agent-driven PRs, together with our own hands-on experience with leading autonomous agents, show that many agent efforts still fail to meet the quality bar of being truly “merge-ready.” They often contain subtle regressions, superficial fixes, or weak engineering hygiene (e.g., [56]). Each failed check demands human review; at agentic scale, this verification work can overwhelm developers. Nevertheless, a new class of practitioners is emerging: developers achieving 100x or even 1,000x productivity. By mastering the nascent practices of this agentic era, these super developers show what is possible. Their success is a powerful proof-of-concept, but it also highlights a familiar challenge.

The core purpose of the SE field has always been to ensure solutions are trustworthy and delivered economically, and much of the SE field exists because we cannot assume that every team is composed of “super developers.” The industry has long acknowledged the phenomenon of the “10x developers,” a small fraction of developers whose impact far exceeds the median [30]. A significant portion of SE, from structured processes like Agile to sophisticated tools like IDEs, is designed to give non-super developers the scaffolding and opportunity to perform at a 10x level. Agentic SE radically reshapes this landscape, moving the conversation beyond 10x to the realm of 100x and even 1,000x productivity while also redefining the characteristics of such top-tier developers, away from raw coding prowess and toward effective collaboration with fleets of agents (aka AI Teammates).

As the industry forges ahead, a Cambrian explosion of ad-hoc practitioner techniques is emerging. However, these grassroots innovations highlight a vacuum of robust, validated approaches. Current methods, relying heavily on informal, conversational prompting, are inadequate for developing trustworthy large-scale, long-lived software. This informality fails to establish robust processes for reproducibility, auditable artifacts for trust, or a durable mechanism for human-agent collaboration. It keeps the paradigm locked in the realm of 1-to-1 “agentic coding,” rather than unlocking the potential of N-to-N “agentic software engineering” where teams of humans and agents collaborate at scale. Early attempts to impose order, like the Plan-Do-Assess-Review (PDAR) loop, are a crucial shift but do not constitute a complete engineering methodology. This new reality demands more than incremental adjustments; it compels us to fundamentally reconsider the pillars upon which the SE field is built: the **Actors**, the **Processes** they follow, the **Tools** they use, and the **Artifacts** they shape.

This call for structure is not unique to software engineering. Parallel debates are unfolding in education, where frameworks for “human-AI co-thinking” are being explored to transform learning [43]. These frameworks emphasize synergistic partnerships, with humans retaining roles of verification and evaluation while treating AI as an intellectual collaborator. Structured Agentic Software Engineering extends this philosophy to engineering, proposing specific structures and disciplines to make such partnerships succeed at scale.

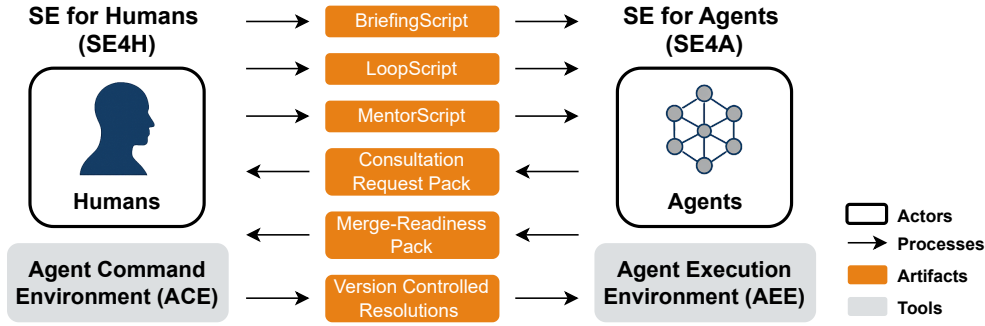


Fig. 1. Structured Agentic Software Engineering (SASE) overview.

This paper proposes a vision for this reconsideration: Structured Agentic Software Engineering (SASE), summarized in Fig. 1. SASE acknowledges that SE is a “wicked problem” where rigid, universal processes are futile. It therefore prioritizes adaptable solutions, arguing that an AI teammate that can be quickly onboarded into the context of a specific team, project or organization is more valuable than a brilliant but brittle specialist agent that falters outside its narrow domain. The core thesis of SASE is the introduction of a structured duality, which posits that the field must simultaneously serve two distinct modalities:

- **SE for Humans (SE4H)**, which redefines the human’s role to focus on high-level intent, strategy, and mentorship as an Agent Coach.
- **SE for Agents (SE4A)**, which establishes a structured and predictable environment where multiple agents can operate effectively.

This duality requires systematically rethinking the four pillars of SE for an agentic era, as they manifest differently across each modality:

- **Actors:** The cast expands from human developers to a hybrid team of human “Agent Coaches” and specialized software agents.
- **Processes:** Ad-hoc prompting gives way to structured, repeatable engineering activities that govern human-agent collaborations.
- **Artifacts:** Transient, informal prompts are replaced by durable machine-readable structured artifacts that serve as contracts and institutional memory, including not only human-authored briefs (BriefingScript) but also agent-generated Consultation Request Packs (CRPs) for invoking human expertise.
- **Tools:** The traditional all-in-one human-centric Integrated Development Environment (IDE) is replaced by specialized workbenches designed for the distinct needs and strengths of humans and agents.

The traditional IDE is ill-equipped for this new era. As the central tooling pillar, SASE proposes two distinct, purpose-built environments.

- The **Agent Command Environment (ACE)** is the command center for the human “Agent Coach.” It is a workbench optimized for human cognition, enabling strategic tasks like specifying intent, orchestrating complex workflows, and reviewing evidence-backed results, while offering full observability into agent activities and associated costs.

- The **Agent Execution Environment (AEE)** is the agents' world, a digital workbench optimized for their unique capabilities, such as high-speed computation, massive parallelism, and tireless, repetitive execution, capabilities that far exceed the limitations of human cognition and stamina.

Rather than a monologue of informal chat, the interaction between these two environments is a structured dialogue carried by explicit, version-controlled artifacts. Humans initiate the dialogue with the **BriefingScript** (mission plan), **LoopScript** (workflow playbook), and **MentorScript** (best-practices guide), but the exchange is not a static handoff. Agents continue it by generating their own formal artifacts: the **Consultation Request Pack (CRP)** to request human expertise and the **Merge-Readiness Pack (MRP)** to present a final, evidence-backed deliverable. Humans then respond with **Version Controlled Resolutions (VCRs)**, auditable artifacts that formally address each CRP or MRP. Versioned updates to these artifacts capture clarification and feedback over time, keeping the shared understanding of tasks, processes, and team norms current. In this way, SASE turns agentic SE from an informal craft into a disciplined engineering practice.

The presented SASE framework is intentionally visionary. Our primary goal is not to offer a definitive solution, but rather to serve as a conceptual scaffold to catalyze an urgent dialogue throughout the SE community. SASE is not a temporary scaffold for current AI limitations. As agents become more capable, the central question shifts from what they can do to what organizations can responsibly delegate. Structured human-agent collaboration remains necessary because software goals embody values, priorities, risks, and stakeholder trade-offs that require accountable human ownership. Therefore, trust must rest on evidence, traceability, and auditable decisions. Research on automation [34] shows that higher levels of automation often shift risk from execution failure to supervision failure, including loss of situation awareness and fragile handoffs.

As autonomous agents become first-class actors in the SE lifecycle, the time has come to re-evaluate the foundational tenets of the SE field. We must look beyond the long-held focus on source code as the canonical artifact and the human as the sole actor, and instead build the new processes and tools that are essential for a collaborative, agentic future. This paper is offered as a first step in that collective rethinking, with the express purpose of shaping the discussions that will define the future of the SE field. The paper culminates in a research roadmap that identifies a few key challenges and opportunities, and briefly discusses the implications for SE education.

2 From Agency to Autonomy: A Hierarchical Framework for AI in SE

To situate the SASE vision within the broader evolution of AI in SE, it is crucial to formalize the progression of intelligent SE (i.e., the integration of AI capabilities into SE). Just as the automotive industry relies on standardized autonomy levels to chart progress in self-driving [40], we need a comparable framework in SE. We first must distinguish between **agency**, defined as the capacity of a system to act and execute plans to achieve a given goal, and **autonomy**, which represents the capacity of a system to self-govern and independently formulate those goals. This distinction allows us to formulate a hierarchical framework, analogous to the Society of Automotive Engineers (SAE) Levels¹ for autonomous driving, that classifies AI capabilities from simple assistance to full automation. Our presented framework below helps to situate and clarify the transition from AI-Augmented SE (SE 2.0) to the Agentic SE (SE 3.0) era that is the focus of this paper.

¹https://www.sae.org/standards/j3016_202104-taxonomy-definitions-terms-related-driving-automation-systems-road-motor-vehicles

Level 0: Manual Coding (No-AI SE) [SE 1.0]

- **Canonical Use Case:** No AI mapping. The human manually translates ideas into tokens by typing.
- **Example Technology Manifestations:** Plain text editors like Notepad, vi and emacs.
- **Car Autonomy Parallel (SAE Level 0):** No Automation. The human performs all driving tasks.

Level 1: Token Assistance (AI-Augmented Coding) [SE 1.5]

- **Canonical Use Case:** Maps a developer’s immediate editing intent to predicted tokens.
- **Example Technology Manifestations:** Standard auto-complete features in all modern IDEs.
- **Car Autonomy Parallel (SAE Level 1):** Driver Assistance. The vehicle features a single automated system for driver support, such as cruise control.

Level 2: Task-Agentic (AI-Augmented SE) [SE 2.0]

- **Canonical Use Case:** Maps a planned code change (e.g., a function description) to a complete, generated block of code. Similar levels of automations for other SE tasks like testing and reviewing code exist.
- **Example Technology Manifestations:** GitHub Copilot, Amazon CodeWhisperer.
- **Car Autonomy Parallel (SAE Level 2):** Partial Automation. The vehicle controls both steering and speed, but the human must constantly supervise and remains responsible.

Level 3: Goal-Agentic (Agentic SE) [SE 3.0]

- **Canonical Use Case:** Maps a technical goal (e.g., “add a caching layer”) to a detailed plan of code changes.
- **Example Technology Manifestations:** Emerging agents like Cognition’s Devin, Anthropic’s Claude Code, Google’s Jules, and OpenAI’s Codex aim for this level. They can take a well-defined goal and execute a multi-step plan (whether self-devised or human-guided) to implement the required changes across code, documentation, and other essential project artifacts.
- **Car Autonomy Parallel (SAE Level 3):** Conditional Automation. The vehicle drives itself under specific conditions, but the driver must be ready to intervene.

Level 4: Specialized Domain Autonomy [SE 4.0]

- **Canonical Use Case:** Maps a broad technical mandate for a specific domain (e.g., “ensure the reliability of the payment service”) to a list of concrete technical goals.
- **Example Technology Manifestations:** This level reflects deep, specialized expertise, a frontier that today’s most advanced LLMs are beginning to target. Specialization typically occurs along two primary axes: the **technical stack** and **quality attributes**. For example, Foundation Models like GPT-5 are now being specialized for the frontend web development domain. As highlighted in GPT-5’s official prompting guide, this involves fusing technical skills (such as “rigorous implementation abilities” with Next.js and Tailwind CSS) with quality attributes like an “excellent baseline aesthetic taste.” Conversely, a Security Agent would specialize along the other axis. It would focus on a single quality attribute (i.e., security) but would be tasked with applying its deep expertise across a diverse range of technology stacks to safeguard all types of software. At level 4, either axis has to be ensured in one domain.
- **Car Autonomy Parallel (SAE Level 4):** High Driving Automation. A Level 4 car is fully self-driving but restricted to a limited operational domain, such as a “geo-fenced” area or, say, specific weather conditions. Similarly, a Level 4

SE agent has high autonomy but only within a particular “technical domain,” be it a specific technology stack or a specific quality attribute domain.

Level 5: General Domain Autonomy [SE 5.0]

- **Canonical Use Case:** Maps a general technical mandate (e.g., “ensure all our systems are robust”) to domain-specific technical mandates for any unfamiliar domain it encounters.
- **Example Technology Manifestations:** The overarching challenge of domain autonomy lies in scaling this fused capability: consistently achieving deep, specialized expertise across the full spectrum of technology domains (e.g., server backends, embedded systems) and quality attributes (e.g., performance, reliability, accessibility, security), transitioning from the domain-specific Level 4. As such, general domain autonomy is currently at the conceptual/research stage, i.e., it does not yet exist.
- **Car Autonomy Parallel (SAE Level 5):** Full Driving Automation. A Level 5 car can travel anywhere on any road, in all conditions. Likewise, a Level 5 SE can apply its high-autonomy capabilities to any technical challenge, regardless of its technology stack and domain, becoming a truly generalized expert.

While this framework outlines a complete trajectory toward the ultimate goal of **Autonomous SE** (Levels 4.0 and 5.0), the immediate, industry-defining challenge lies in mastering the **Agentic SE** era (aka SE 3.0). The transition from Level 2.0 to Level 3.0 represents more than an incremental step; it fundamentally shifts the human-computer relationship, introducing immense complexity in workflow orchestration, trust, and verification. Before the community can realistically pursue full autonomy, we must first establish the disciplined practices required to manage goal-agentic systems. Therefore, the SASE vision presented in this paper is focused squarely on the artifacts, processes, and tools necessary to successfully engineer trustworthy software within the SE 3.0 (aka Agentic SE) era.

3 The Emergence of Agentic Software Engineering

3.1 Industrial Relevance

The SE field has emerged as a primary proving ground for demonstrating the return on investment (ROI) of large-scale generative AI models like Large Language Models (LLMs). This strategic focus by frontier AI labs and the broader industry is motivated by a unique convergence of factors:

- (1) **High-Cost Workforce:** Software engineers command premium salaries, meaning even modest productivity gains among this workforce can translate into substantial financial returns.
- (2) **Rich Training Data:** Code repositories, issue tickets, and commit histories constitute one of the most extensive and well-structured datasets of any knowledge-work domain (a fact the Mining Software Repositories (MSR) community has recognized for decades [18, 20]).
- (3) **Measurable Outcomes:** The field offers clear, quantifiable success metrics (e.g., compiler errors, test outcomes, defect rates) that are critically valuable for creating effective and robust reward functions for reinforcement learning (RL) [29, 50].
- (4) **Robust Safety Nets:** The presence of automated testing and CI pipelines mitigates the risk that such models will fail in practice.
- (5) **Transferable Benefits:** Foundation Models honed on SE workflows [25] generalize well to other business tasks that lack equivalent guardrails and data. This powerful combination of factors has, in turn, ignited a cutthroat competition among several frontier companies, all racing to define and dominate the nascent market for Agentic SE.

This concerted industrial focus has rapidly transitioned **Agentic Software Engineering**, corresponding to the **Goal-Agentic (Level 3)** stage of our framework, from a theoretical concept to a boardroom-level strategic imperative. This is evidenced by the release of specialized coding agents by all leading players (e.g., **Google’s Jules**, **OpenAI’s Codex**, and **Anthropic’s Claude Code**) and a flurry of strategic mergers and acquisitions aimed at securing the invaluable stream of developer feedback data generated from AI-native development environments. This competitive landscape underscores the industry’s intense focus on securing the SE data flywheel, which comprises both a user base and the associated feedback data needed to refine the next generation of SE-focused LLMs [23, 47].

3.2 What is an Agent and Key Recent Observations

Despite the high stakes of Agentic Software Engineering, the fundamental concept of an “agent” remains loosely defined. To bring clarity, we situate different agent implementations on a spectrum defined by **agency** (executing a given plan) and **autonomy** (formulating the plan itself). This mapping helps clarify the distinction made by frameworks like Anthropic’s²:

- **Workflow Agents (High Agency):** These are predefined orchestrations of LLMs and tool invocations. Because their high-level logic and flow are hardcoded by a human developer, they primarily exhibit **agency** by executing a given plan to achieve a goal. While low-level tasks within the plan might be handled with some autonomy, the overall system is not self-governing.
- **Autonomous Agents (High Autonomy):** These are systems where agents are given a high-level goal and exhibit **autonomy** by planning, reasoning, and invoking tools to formulate their own path to completion.

This distinction is critical. Workflow agents, as systems of agency, require ongoing manual updates to their core orchestration logic to adapt. In contrast, autonomous agents can be iteratively guided and improved by human developers through natural language. This latter approach eliminates low-level code rewrites and enables flexible adaptation, representing a form of “FMware” that is coded and rewired using English prose [10]. This marks a fundamental shift in how we build software: from explicitly coding logic to declaratively describing behavior.

3.3 Brief Survey of Today’s Agentic Solutions on Benchmarks like SWE-Bench

A critical insight emerging in the Agentic SE space is that passing tests alone is no longer enough. Recent deeper examinations of SWE-Bench results [24] (the de facto benchmark for evaluating Foundation Model capabilities in SE) highlight a key limitation. The code generated by today’s Foundation Models is still far from being merge-ready for professional codebases:

- 29.6% of “plausible” fixes introduced behavioral regressions or were incorrect upon rigorous retesting [48].
- True solve rates for GPT-4 patches dropped from 12.47% to 3.97% after detailed manual audits, revealing widespread weak or cosmetic solutions [3].
- AI agents frequently produced superficial patches limited to single files, unlike human developers [5].
- Many patches passing unit tests failed broader CI checks due to style or hidden regressions [54].

SWE-Bench Verified [31] was introduced to address concerns that some SWE-Bench tasks were ambiguous or underspecified. OpenAI, working with the authors of SWE-Bench, released SWE-Bench Verified as a human-validated subset of 500 tasks. Results on this subset showed rapid progress: GPT-4o solved 33% of tasks at the August 2024 launch, while leading agentic solutions exceeded 70% by mid-2025. Yet the benchmark has also become harder to interpret.

²<https://www.anthropic.com/engineering/building-effective-agents>

OpenAI later warned that SWE-Bench Verified was increasingly exposed to data contamination and recommended SWE-Bench Pro [11, 32] as a more reliable evaluation. The progression from SWE-Bench to SWE-Bench Verified and then to SWE-Bench Pro therefore illustrates both the field’s rapid progress and the difficulty of maintaining uncontaminated measures of agent capability.

These benchmark shifts emphasize that passing tests, even on verified tasks, is not enough. Even in mature, production-grade ecosystems like the .NET runtime, developers are observing the same pattern: **passing tests is far from sufficient**.³ Achieving merge-ready status requires a deeper understanding of context, intent, and the broader system. These are qualities that today’s agents still struggle to demonstrate reliably.

3.4 Brief Survey of Today’s Agentic Solutions in the Wild using GitHub Data

Projections from industry leaders, including Google’s Chief Scientist Jeff Dean,⁴ suggest that AI agents will soon perform at the level of junior developers. One early study [49], which focuses on Claude Code, found that agent-assisted contributions were commonly used for refactoring, documentation, and testing activities. Notably, 83.8% of these pull requests were eventually merged, and more than half of the accepted contributions were integrated without additional modification.

Recent large-scale studies of GitHub repositories indicate that agentic coding is already widespread. For example, the AIDev dataset [27] contains 932,791 agent-authored pull requests (PRs) generated by tools such as OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code across 116,211 repositories (dataset cutoff: August 1, 2025), providing one of the first large-scale views of agent participation in software development. Based on AIDev, a large-scale study of 15,451 refactoring instances across 12,256 agent-authored PRs found that agents frequently perform localized and consistency-oriented refactorings, such as variable renaming and type updates, while undertaking fewer high-level architectural changes than human developers [22].

At the same time, several studies highlight important limitations. An analysis of 2,303 agent context files (“Agent READMEs”) from 1,925 repositories found that developers primarily provide agents with functional guidance, such as implementation details, architectural information, and build instructions, while specifying security and performance requirements far less frequently [7]. Similarly, a study of 4,550 agent-authored pull requests reported that agents often handle logging and observability inconsistently, with human developers subsequently correcting many of these issues during review and integration [33].

4 Motivational Example: The Anatomy of an Agentic SE Workflow

This section grounds the Agentic SE era in a concrete example: a developer resolving seven distinct pull requests in a production codebase. This workflow (as shown in Fig. 2) provides a lens through which we can observe both the impressive new capabilities offered by this era and the underlying process and tooling challenges that SASE is designed to solve.

4.1 The New Workflow: A Glimpse into Agentic SE in Practice

In this scenario, the developer’s role shifts from coder to specifier. Instead of writing code for each ticket, they spend approximately 1.5 hours authoring detailed, natural-language specifications and guidance for each of the seven tickets. These specifications then trigger a team of autonomous agents to work asynchronously, generating 28 distinct pull

³https://www.reddit.com/r/ExperiencedDevs/comments/1krttqo/my_new_hobby_watching_ai_slowly_drive_microsoft/

⁴<https://www.businessinsider.com/google-boss-ai-junior-coder-within-a-year-2025-5>

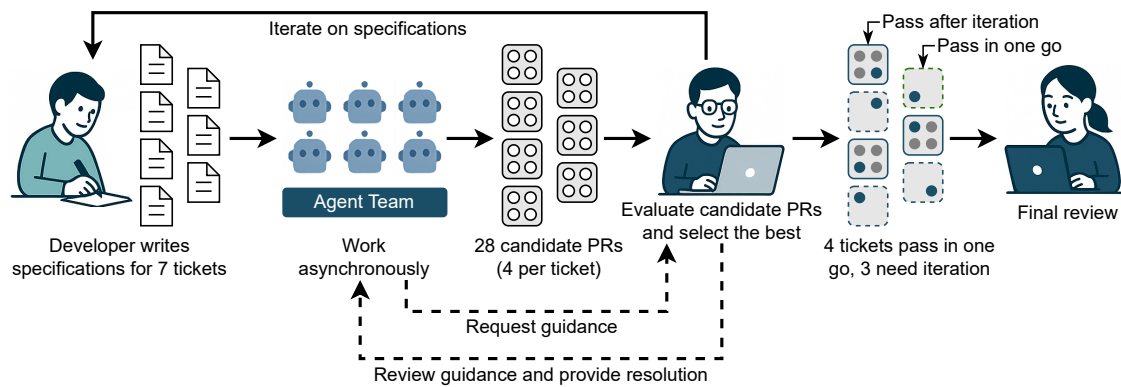


Fig. 2. Overview of an agentic SE workflow

requests in parallel (4 pull requests per ticket). This highlights the re-emergence of N-version programming [6, 8, 28, 39], a powerful practice that not only serves as a form of inference-time compute to increase the probability of a successful outcome through trial and error but also enables creative exploration.

The developer then evaluates the different solutions, selecting the most promising one if pull requests satisfy the initial natural-language specification or refining the latter if none of the four pull requests is acceptable, followed by re-triggering the agents. Once all tickets have yielded an acceptable solution, the latter are submitted for review and (eventually) are approved and merged into the code base.

4.2 The Process and Artifact Gaps

This new workflow exposes critical gaps in current SE processes and the artifacts that are used to support them:

4.2.1 The Art of the Briefing: From Vague Tickets to Actionable BriefingScripts. A common failure pattern when using AI agents is to simply paste a raw ticket and expect magic. The informal, natural-language tickets are a source of ambiguity. A more rigorous process would treat this specification as a first-class artifact, moving towards structured specifications where the degree of formality can be adapted to the task at hand. The 100x/1000x developers treat an agent as a junior team member or an outsourced partner rather than a magical tool. They excel at providing a comprehensive initial briefing that includes not only the specification but also the bigger picture, relevant context, and strategic advice on how to break down the task and how to go about doing it.

To formalize this crucial skill, we advocate for moving beyond ad-hoc prompts toward a structured artifact we call a **BriefingScript**. This is much more than a specification of intent; it is the detailed work order that a senior developer would give a junior one to ensure success, complete with:

- **What & Success Criteria:** Defines the scope with a verifiable checklist, similar to Scrum’s “Definition of Done,” but enriched with formal, testable properties like pre-conditions and invariants.
- **Architectural Context:** Clarifies where the work fits in the system, identifying key modules, data models, or APIs to interact with.
- **Strategic Advice:** Recommends specific implementation approaches, such as libraries to use or patterns to avoid, guiding the agent’s problem-solving strategy.

- **Potential ‘Gotchas’:** Highlights known pitfalls or tricky areas to watch out for, like subtle business logic, performance constraints, or dependency issues.

Crucially, a BriefingScript is not a rigid, one-shot specification. Like pair programming or collaborative design, it evolves through iterative dialogue between the human coach and the agent. Early drafts may be lightweight, progressively enriched with clarifications and refinements based on agent feedback. This iterative style avoids the brittleness of upfront, waterfall-style specifications and reflects how elite engineers already operate in practice. This approach aligns with broader work on human-AI co-thinking [43], where humans act as verifiers and evaluators while AI provides generative power. By codifying guidance into BriefingScripts, we ensure that agents operate with both clarity and accountability. In addition, the BriefingScript must be a living document that evolves over time. Subsequent feedback and clarifications between humans and agents must be incorporated back into the BriefingScript as versioned updates. This approach directly mirrors the principles of managing institutional knowledge in large-scale software engineering. As detailed by Hyrum Wright and others in Software Engineering at Google [52], elite engineering organizations rely on shared, version-controlled knowledge bases as a durable source of truth for human developers. By applying this same proven principle to human-agent collaboration, the BriefingScript transforms from an initial set of instructions into an evolving, auditable record that always reflects the complete and current shared understanding of the task.

We can make this process more robust by creating version-controlled BriefingScripts. Rather than a new programming language, a BriefingScript is a structured, machine-readable artifact that may be serialized in formats such as Markdown, YAML, JSON, or a domain-specific schema. This approach can be viewed as a modern, agent-oriented evolution of Donald Knuth’s concept of “literate programming.” Instead of treating code as the primary artifact, the focus shifts to the human-readable BriefingScript: a document that explains the logic and intent from which the agent’s work is derived. Looking ahead, creating high-quality BriefingScripts is a significant skill of the elite software engineer, and AI-powered tools could greatly assist developers in drafting them. As an agent becomes more attuned to a codebase and its human collaborator(s), the explicit, human-drafted portion of these scripts should shrink, making the entire SE process more efficient. A concrete example of a BriefingScript is provided in Appendix A.1.

4.2.2 The Multidimensional Nature of Agentic Feedback and Mentorship. In the agentic SE era, the feedback and mentorship loop becomes significantly more complex than traditional code review. Guidance is not limited to the final code but extends to the entire SE process, encompassing both explicit instructions and implicit principles that the agent must infer. Key dimensions of this new feedback model include:

- **Explicit & Durable Mentorship:** Direct, generalizable guidance from a human coach (e.g., “Avoid obvious comments; instead, comment on the design rationale”) must be captured durably. This prevents the agent from repeating mistakes and is the motivation for **MentorScript**, a version-controlled rulebook that codifies best practices.
- **Inferred Mentorship:** Not all guidance is articulated as a general rule. An agent must be able to infer a broader principle from a specific contextual correction by a human, enabling it to learn instead of being spoon-fed. For example, after a human refactors a piece of code for better readability, the agent should propose a new, general rule about that pattern for the coach to approve and add to MentorScript.
- **Holistic Process Feedback:** Mentorship extends beyond the code to encompass the entire SE lifecycle. A human coach may provide feedback on the agent’s problem-solving approach, its test planning strategy, the way it debugs or fixes build issues, or its choice and use of tools.

- **Feedback on Multiple Solutions:** The ability of agents to generate multiple potential solutions (*N*-versions) introduces novel feedback patterns. A coach’s guidance might involve synthesizing a final solution from different drafts, such as instructing an agent to “combine the UI from solution 1 with the backend logic from solution 2.”

4.2.3 From Ambiguous Control to Explicit Orchestration. As observed by industry leaders like Andrej Karpathy,⁵ agents cannot infer the expectations or “stakes” of a task; they may “overthink” a simple request or under-deliver on a critical one. For some tickets, a coach may want to grant the agent full autonomy, while for others, they might wish to enforce a strict process. This process can be defined at an organizational level by process engineers (e.g., for regulatory purposes) and optionally overridden by the coach for a specific ticket, with the override being recorded. Today, this is handled via ad-hoc prompt hacking in the master prompt for these agents.

This motivates the need for **LoopScript**, a declarative language for defining the agent’s workflow, allowing a coach to explicitly communicate the required level of rigor and enforce a precise Standard Operating Procedure (SOP) when needed. We are already seeing this happen in the frontier Autonomous Coding Agents: Some agents, like Google Jules, have included a planning step from their inception. Others, like Claude Code, have only recently added an on-demand planning mode in which the agent generates a plan and awaits human review before proceeding.

4.2.4 From Code Review to Evidence-Based Oversight. The ultimate goal is to produce a merge-ready contribution. Rather than reviewing dozens of raw pull requests, a human reviewer should focus on auditing a structured **Merge-Readiness Pack**. This is a bundle of evidence designed to bridge the critical gaps between current agent outputs and the standards of a truly merge-ready contribution. The pack proves the agent’s work is trustworthy by providing clear evidence for five key criteria:

(1) Functional Completeness. *The Gap:* Agents often produce superficial or partial fixes that pass a narrow set of tests but fail to address the holistic user need. *The Evidence:* The pack must provide proof (e.g., end-to-end test results) that the feature is complete and behaves as specified in realistic scenarios.

(2) Sound Verification. *The Gap:* Agents may generate code that passes an existing, weak test suite, or fail to create new, robust tests for their own logic. *The Evidence:* The pack includes not just passing test logs, but the agent’s test *plan* and the new test cases it generated, proving the verification strategy itself is sound.

(3) Exemplary SE Hygiene. *The Gap:* Agent-generated code can be functional but difficult to maintain, often violating project style guides or principles (e.g., DRY, SOLID). *The Evidence:* The pack includes reports from static analysis, linting, and complexity checkers to demonstrate that the code is clean, readable, and minimizes technical debt.

(4) Clear Rationale and Communication. *The Gap:* An agent’s reasoning is often buried in low-level, verbose trajectory files or chat logs that are impractical for a human to audit. *The Evidence:* The pack synthesizes this into a clear, human-readable summary (analogous to a PR description) explaining the approach and trade-offs.

(5) Full Auditability. *The Gap:* True reproducibility is a major challenge due to agent non-determinism or environment changes. *The Evidence:* The pack provides a “frozen” audit trail, including versioned links to the exact BriefingScript/MentorScript, tools, and agent trajectory used, ensuring the result can be reliably reproduced.

To manage this density of information, the pack must support “**progressive disclosure**,” allowing a reviewer to see a high-level summary and then drill down into specific evidence like test logs or execution traces as needed. A concrete example of an MRP is provided in Appendix A.3.

⁵<https://x.com/karpathy/status/1954224651443544436?s=46&t=ayaQZ2-uUhb4rg402jW5w>

4.3 The Tooling Gaps

Finally, this motivational example highlights a fundamental mismatch between the new agentic workflow and the tools that are available for both humans and agents:

4.3.1 A New Workbench for the Human Developer: The ACE. The traditional Integrated Development Environment (IDE) is ill-equipped for the new era of agent-assisted SE. Today’s AI-IDE tools like Cursor remain too code-centric and have yet to treat mentorship, which involves engineering artifacts beyond code, as a central activity. Human developers acting as coaches therefore need a command center for orchestrating parallel agent work. We call this workbench the **Agent Command Environment (ACE)**. The ACE supports both 1-to-N collaboration, where one developer works with many agents, and N-to-N collaboration, where a human team coordinates a shared fleet of AI teammates.

This team-level setting requires multi-role governance, evidence-based review, and cross-functional consultation. Consultation Request Packs (CRPs) operationalize that collaboration: an agent can invoke a human specialist through a CRP, and the ACE routes, presents, and records the request. In this sense, the ACE treats humans as callable expertise endpoints while preserving the context needed for accountability. A concrete example of a CRP is provided in Appendix A.2.

The ACE must integrate capabilities that are currently missing from standard development tools. It should support disciplined N-version programming, allowing a developer to visualize, compare, and mix components from multiple agent-generated solutions. It should also provide program-comprehension views that show the architectural impact of generated changes, moving beyond simple textual diffs. Because SASE relies on explicit artifacts, the ACE must support authoring, versioning, archival, and analysis for BriefingScript, MentorScript, and LoopScript. It must also help coaches curate the complex context agents need, which often differs from the context humans need. Finally, the ACE should support strategic agent management: coaches should be able to compose an agent team based on capability and cost, evaluate performance, and retrain, demote, or retire underperforming agents. The ACE must also let the coach “jump in” when direct implementation is more efficient than specification. For example, a developer should be able to switch into a traditional IDE view for a surgical code change, such as writing a complex mathematical formula, and then return to the coaching workflow.⁶

Voice can serve as a complementary interaction modality for the ACE. It may be useful for high-level orchestration and mentorship tasks, especially when a developer needs to issue brief commands, dictate intent, or provide feedback without leaving the current work context. Research has shown that (a) speech can be faster than typing [15]; (b) voice-assisted debugging can reduce context switching in software development workflows [4]; and (c) current automatic speech recognition (ASR) systems, such as OpenAI’s Whisper [37], make reliable transcription increasingly practical. The ASR layer need not perform reasoning; it can capture developer intent and pass the resulting text to downstream ACE tools and agents. Developer tools such as Talon Voice⁷ and Cursorless for VS Code⁸ further demonstrate the feasibility of speech-based interaction with agents in accessibility and hands-free settings.

4.3.2 An Optimized Workbench for the Agent: The AEE. Just as humans need a new command center, agents require their own specialized environment designed for their unique capabilities. The tools that excel for humans are often suboptimal for agents. Much of modern SE has focused on creating high-level tools that reduce cognitive load for humans. This

⁶The demo provides an example of performing a surgical code change when developing mobile apps: https://www.linkedin.com/posts/ahmed-c-hassan_se3-harmonyos-ainative-activity-7274452304278265856-7F7L

⁷<https://talonvoice.com>

⁸<https://www.cursorless.org>

optimization, however, is often at odds with the needs of an agent. Agents, unburdened by human cognitive limits, thrive on raw, low-overhead tools that are optimized for computational efficiency and provide structured, machine-readable feedback. That many of today’s autonomous coding agents still rely on basic utilities like `grep` exposes this fundamental mismatch.

This necessitates the creation of an **Agent Execution Environment (AEE)**: a workbench built for agents, not humans. Instead of human-centric interfaces, the AEE must be equipped with agent-native tools that leverage their strengths. These tools might include hyper-debuggers capable of analyzing vast state spaces, powerful semantic search utilities, and structural editors that manipulate code as abstract symbolic structures rather than simple text. For example, early ideas considered structural manipulation through abstract syntax trees [35], while a more robust paradigm focuses on symbolic reasoning and formal environments [46]. Beyond these task-oriented tools, the AEE must also include a robust monitoring infrastructure to manage the agents’ operational health. This internal system would autonomously handle low-level issues such as spotting security vulnerabilities, flagging agents that are incurring unexpectedly high computational costs, or repairing and replacing broken virtual environments. The goal of this self-monitoring is to ensure that only significant problems requiring strategic human intervention are surfaced to the human in the ACE.

5 The Engineering Activities of SASE

SASE is operationalized through structured engineering activities. This section does not present a definitive or exhaustive list; instead, it offers an initial scaffold for organizing N-to-N collaboration among human coaches, AI teammates, and hybrid teams. The activities distinguish team-level agentic software engineering from solo agentic coding by making governance, evidence-based review, cross-functional consultation, and merge-readiness explicit. Consultation Request Packs (CRPs), for example, turn human consultation into a traceable team artifact. We invite the community to challenge, refine, and extend these activities as Agentic SE matures.

5.1 Briefing Engineering (BriefingEng): The Art of the Mission Briefing

In the Agentic SE era, the primary creative output of an engineer evolves from implementation logic to the articulation of unambiguous intent and guidance. Briefing Engineering (BriefingEng) is the activity that codifies this crucial skill. This activity does not seek to reinvent the wheel; instead, it builds upon the decades of foundational work from the Requirements Engineering (RE) and Agile/Scrum communities, adapting their principles for an agentic context. It is a hybrid discipline that fuses requirements specification with architectural design, strategic implementation advice, and test planning into a single, cohesive artifact. As the “brief” becomes as critical as, if not more critical than, the code itself, we see a vital opportunity for researchers and practitioners from these communities to lead the charge in co-designing the next generation of specification practices for a future where humans guide and agents build.

Purpose. This activity moves beyond the common failure pattern of pasting a raw, vague ticket and expecting magic. It treats the mission brief as a first-class artifact, ensuring an autonomous agent receives a comprehensive and actionable work order. Unlike a traditional Software Requirements Specification (SRS), which is often implementation-agnostic, a BriefingScript is a specification for action: a version-controlled, testable, and machine-readable document that is as central to the engineering process as the source code itself.

Actor. The human Agent Coach.

Workbenches. All BriefingEng activities are centered in the Agent Command Environment (ACE). AI assistance can be used to help the coach author high-quality briefs by flagging ambiguity, surfacing edge cases, ensuring logical consistency, and generating property-based acceptance tests for the final output from the briefs.

Artifacts. The BriefingScripts. A BriefingScript is a structured, version-controlled artifact that may be serialized in any machine-readable format (e.g., Markdown, YAML, or JSON), making the specification itself traceable and reviewable. While its structure provides discipline, it is not intended to enforce a rigid, up-front specification. Instead, BriefingScripts are best authored through an interactive, iterative process, where the initial draft may be lightweight and refined over multiple cycles of agent interaction. This flexibility ensures briefs evolve naturally alongside the task, rather than locking humans and agents into an unrealistic “waterfall” style process. While the RE field has long pursued this goal, it becomes more feasible in the agentic context for three reasons:

- (1) The primary consumer is a machine, which necessitates and benefits from a formal structure;
- (2) Briefs are often for more granular tasks than a monolithic SRS, making formalization tractable;
- (3) Modern AI assistants can help humans write these structured briefs, lowering the barrier to entry that hindered past formal methods.

Emerging industry examples of BriefingScript-like artifacts, such as the Product Requirement Prompt (PRP), are discussed in Section 8.1.

5.2 Agentic Loop Engineering (ALE): Disciplined Orchestration

With a clear brief in place, Agentic Loop Engineering (ALE) governs how agents execute tasks. Just as BriefingEng builds on the work of RE and Agile, ALE is deeply rooted in the principles pioneered by the DevOps community. It transforms the agent’s work from an opaque, black-box process into a disciplined, auditable, and reproducible workflow, moving beyond simple iterative cycles like the Plan-Do-Assess-Review (PDAR) loop. The declarative pipelines, infrastructure-as-code, and focus on observability that are central to modern DevOps are the direct precursors to the automated, agent-driven workflows defined by a LoopScript. We therefore see a crucial role for the DevOps community in designing the next evolution of CI/CD.

Purpose. This involves defining how agents work together (or alone), the patterns of their collaboration, and how they engage with their toolset. Such explicit direction is essential because agents cannot infer the “stakes” of a task; they may “overthink” a simple request or under-deliver on a critical one.

Actors. The workflow is orchestrated by the **human coach**, but the execution is performed by **agents**.

Workbenches. The workflow is defined in the **ACE**, but the agents execute it within the **AEE**.

Artifacts. The LoopScripts. Instead of relying on ad-hoc prompt hacking, the coach uses a declarative language like LoopScript to define the Standard Operating Procedure (SOP). Like all SASE artifacts, the LoopScript is a living document. A coach might dynamically adjust the workflow (for instance, by allocating more agents to a promising path or by adding a new review checkpoint if early results look uncertain) ensuring that the orchestration strategy is customized for a task when needed. A LoopScript can specify:

- **Task Decomposition and Parallelization:** A BriefingScript can be assigned to multiple agents, which could involve multiple instances of the same model or, more powerfully, a heterogeneous team composed of specialized agents. This reflects the emerging practice of using different models for their distinct strengths, for example, using a model like Gemini 2.5 Pro for high-level planning while leveraging Claude Opus or Sonnet for the detailed code generation. This makes powerful practices like N-version programming routine. For instance, a developer resolving seven tickets can trigger such a team to generate 28 distinct pull requests in parallel (4 per ticket), enabling creative exploration and increasing the probability of a successful outcome. The key metric shifts from single-task latency to overall system throughput.

- **Workflow Strategy:** The coach can define the required level of rigor, granting full autonomy for a simple bug fix while enforcing a strict, multi-stage review process for a critical security patch.
- **Evidence-Based Acceptance Criteria:** The LoopScript defines the structure of the final deliverable: a “Merge-Readiness Pack.” As detailed in Section 4.2.4, this bundle proves the agent’s work meets the five core criteria for being merged. Emerging examples of LoopScript are discussed in Section 8.1.

5.3 AI Teammate Mentorship Engineering (ATME): Codifying Team Norms and Best Practices

To ensure agent-generated code is not just functional but also maintainable and aligned with team culture, agent guidance must be treated as first-class code.

Purpose. To transform mentorship from an implicit, ephemeral activity (e.g., comments in a code review) into an explicit, evolving, and codified discipline. This is “mentorship-as-code.”

Actors. Guidance is provided by the **human coach** and durably consumed by **agents**.

Workbenches. Mentorship rules are authored in the **ACE** and directly influence agent behavior in the **AEE**.

Artifacts. The **MentorScripts**. These are structured, machine-readable rulebooks that codify project norms (aka tribal knowledge and aligned understandings). A MentorScript allows teams to define rules ranging from granular checks (“all new functions must have deterministic tests”) to high-level principles. This makes mentorship, once an implicit and ad-hoc activity, an explicit, reviewable, and continuously evolving discipline. MentorScript rules would be subject to their own quality gates, including linting, unit testing, and conflict detection, ensuring they are atomic and deterministic. Crucially, every action an agent takes is traced back to the MentorScript rules that were considered (through prompt interpretation techniques such as PromptExp [12] and reasoning observability techniques such as Watson [38]), enabling rapid root-cause analysis when the behavior of an agent deviates from expectations. This explicit guidance reduces the burden on the human coach to infer complex rule interactions, making the behavior of agents more predictable and reliable.

Structured Mentorship. The review process generates a critical, multidimensional artifact. When a coach provides feedback, it is captured systematically across the explicit, inferred, and multi-solution dimensions previously outlined in Section 4.2.2.

An early grassroots example of MentorScript-like artifacts, the use of meta-prompt files (e.g., CLAUDE.md, AGENT.md), is discussed in Section 8.1.

5.4 Agentic Guidance Engineering (AGE): Leveraging the Human in the Loop

While BriefingEng initiates work, **Agentic Guidance Engineering (AGE)** governs the structured role of the human in reviewing and responding to agent-generated artifacts and clarification requests (e.g., during the BriefingEng and Mentoring activities). AGE elevates the human from a passive approver of outputs to an active, on-demand consultant who intervenes precisely where their expertise adds the greatest value.

Purpose. To formalize and optimize human participation in the agentic loop, ensuring that when agents escalate issues through a Consultation Request Pack (CRP) or submit a Merge-Readiness Pack (MRP), human input is efficient, targeted, and becomes a durable part of the project record.

Actors. The human engineer (who may be the task initiator or a domain specialist).

Workbenches. All AGE activities are performed within the **ACE**, which provides an inbox-like interface for triaging CRPs, auditing MRPs, and issuing structured resolutions.

Artifacts. AGE consumes two types of agent-generated artifacts: (a) **Consultation Request Packs (CRPs)** and (b) **Merge-Readiness Packs (MRPs)**, and produces **Version Controlled Resolutions (VCRs)**. CRPs are generated when an agent requires human input to proceed. A CRP is contextualized by the active BriefingScript, potentially triggered by LoopScript or MentorScript rules, and documents the specific uncertainty or decision point. MRPs are structured evidence bundles submitted for human approval, designed to demonstrate functional completeness, sound verification, engineering hygiene, rationale, and auditability. The outcome of AGE activities is a Version-Controlled Resolution (VCR). Each Resolution is explicitly linked to the artifact that it addresses (CRP or MRP), preserving traceability and enabling downstream auditing and learning.

5.5 AI Teammate Lifecycle & Infrastructure Engineering (ATLE & ATIE): Building the SE for Agents Foundation

To fully unlock the agentic SE, we must simultaneously support the SE activities of agents (i.e., SE for Agents). This involves fundamentally rethinking our tools, practices, and artifacts when the primary actor is an agent, not a human. The best practices that have served us for decades, designed around the constraints of human cognition and patience, must now be re-examined and, in many cases, inverted. Engineering this new, agent-centric foundation is a monumental task that falls squarely within the expertise of the Platform Engineering community. As agents begin to operate at scale, the need for the robust, secure, and scalable platforms that are the core mission of Platform Engineering becomes paramount. Their work is essential in building the next generation of internal developer platforms, not for humans, but for the fleets of autonomous agents that will inhabit them, ensuring these systems are reliable, efficient, and secure.

Purpose. To engineer the agent’s environment (the AEE), enable agents to retain memory and learn over time (ATLE), and build the agent-native toolchains they need to operate effectively (ATIE).

Actors. The fundamental shift is in the actor itself. For decades, SE has been optimized for the human developer. In an agent-centric world, the primary actor is **computational**. The human engineer’s role evolves into that of a strategist, mentor, and conductor, serving as the ultimate arbiter of value and the indispensable conduit for tacit, “tribal” knowledge. The ultimate goal is not agents that are perfect out of the box but ones that can learn and ramp up first.

Workbenches. This work is fundamentally about architecting the **Agent Execution Environment (AEE)**.

Core Concepts. ATLE and ATIE are grounded in several foundational concepts that collectively define the software engineering infrastructure required for agent-centric development. These concepts span memory, coding practices, tooling, team organization, and long-term agent evolution.

(1) Persistent Memory (ATLE): Agents are embedded with long-term memory of project history and decision logs, allowing them to maintain continuity across tasks without the coach repeatedly supplying the same guidance.

(2) Agent-First Code Practices (ATLE): With the agent as the primary actor, long-standing process principles must be re-evaluated. For instance, the “**Don’t Repeat Yourself**” (DRY) principle is often reversed. Code cloning, a source of maintenance debt for humans, can become a viable strategy for an agent, as it simplifies its reasoning process while the downside (updating all instances) is trivial. This theoretical shift is supported by industry observations, such as the GitClear report noting sharp increases in code duplication on GitHub since the emergence of Copilot. Furthermore, the **ROI of Clean Code** (high cohesion, low coupling, comprehensive documentation) becomes crystal clear, as these practices make a codebase a more fertile environment for agents to inhabit. Such efforts are analogous to making an open-source project accessible to novices through a well-defined plugin architecture, in turn attracting and growing a large community around that project. Previously, this meant redirecting resources away from immediate business needs like feature development. Now, a small human team, aided by agents, can perform this cleanup, which in turn

unlocks massive productivity gains for its fleet of feature-developing agents. This new model also favors programming languages with strong, compile-time safety guarantees, such as *Rust* and *TypeScript*. The up-front effort to satisfy a strict compiler is less of a barrier for an agent, and the payoff is immense, as the strong type system prevents entire classes of bugs by construction.

(3) Agent-Native Toolchain (ATIE): The tools built for human developers are often ill-suited for agents. We have spent decades building tools like IDEs and visual debuggers to reduce cognitive overload, but an agent has no such limitations. This shifts the entire optimization landscape. SE for Humans has historically focused on precision@K for a small K, because human time is precious. In an agent-first world, precision@100 is perfectly acceptable if a subordinate agent can post-process the results, opening up entirely new avenues for automated analysis where the human is completely out of the loop.

Expressive feedback is paramount. Rust’s toolchain exemplifies this, as its rich, constructive compiler messages enable agents to learn quickly from failures, providing a blueprint for agent-friendly environments. The path forward involves creating Agent-Native Model Context Protocol (MCP) servers [17] that return deep, interpretable feedback and support agent-driven refinement of tool usage descriptions. This kind of self-improving tooling loop, already being adopted manually by teams like Anthropic by optimizing their MCP descriptions to suit Agents instead of Humans, is proving essential for making agentic SE robust, scalable, and fault-tolerant.

(4) Engineered Multi-Agent Teams: Paradoxically, while some human-centric rules are inverted, foundational principles for managing system complexity, like Modularity and Separation of Concerns, become even more critical. This is driving a key trend away from monolithic, general-purpose agents and toward engineered multi-agent teams. These systems are structured workflows where agents are assigned specialized roles like “planner,” “coder,” “tester,” and even a “critic” to create an internal feedback loop.

While this specialization is a practical solution to the core limitations of today’s large models (protecting each agent’s context from the “pollution” of side-tasks and an overwhelming number of tool choices), it also points to a more robust, long-term architectural principle. A modular design improves the interpretability of the system’s behavior, as the trajectory of a specialized agent is far easier to audit than the interleaved reasoning of a monolithic one. This also paves the way for an ecosystem with highly specialized agents. This, in turn, could lead to the emergence of “agent stores”: digital marketplaces analogous to today’s app stores. In such a future, a bespoke team could be dynamically composed, with a coordinating agent (whether human or AI) selecting best-in-class agents from various vendors for specific roles, such as a “React Refactoring Agent” or a “Python Security Audit Agent.” Moreover, Multi-Agent Teams provide critical security and control benefits. By assigning specific tools as well as security and resource-usage policies to specialized agents (e.g., a “Test-Agent” that can run tests but not commit code), the system limits the potential “blast radius” of a misbehaving or compromised agent.

These observations are already manifesting in different ways. For instance, the evolution of commercial Autonomous Agents like Anthropic’s Claude Code has shifted from a monolithic agent to a multi-agent architecture where specialized sub-agents are spawned for specific tasks. A more comprehensive example is the BMAD (Breakthrough Method for Agile AI-Driven Development) framework, which takes the “team” metaphor literally, organizing agents into a full-fledged agile structure to tackle complex projects.

(5) Lifetime Teammates: Perhaps the most significant shift is moving from stateless agents to persistent teammates that learn and grow over time. This is the focus of **AI Teammate Lifecycle Engineering (ATLE)**, a discipline aimed at giving agents memory and long-term context.

- **From Contractors to Partners:** The goal is to evolve agents from “one-off contractors” who start every new task from scratch to “life-long partners” who retain institutional knowledge. Early examples of this concept, like the **DeepWiki** used by the Devin agent, allow an agent to build and refer to its own documentation and decision logs across multiple tasks, creating continuity and preventing it from repeating mistakes.
- **Proactive Maintenance:** An agent with persistent memory and access to the codebase can become a proactive partner. During idle compute cycles, the ACE can schedule agents to perform valuable maintenance tasks, such as scanning for technical debt, identifying documentation gaps, or proposing code refactorings. These proposals would be filed as new BriefingScripts, entering the standard SASE workflow for human review and prioritization, thus transforming maintenance from a reactive chore into a continuous, autonomous improvement process and eventually moving SE 3.0 into SE 4.0, where we remove the requirements to get human approval on such optimization activities. A comprehensive industry example of these principles is the BMAD framework, discussed in Section 8.2.

6 Discussion

6.1 The Critical Gap: Observability, Archival, and Revision Control

Despite the rapid, practitioner-driven innovations, the current tooling landscape reveals critical gaps in the foundational pillars of SE. The current agentic offerings are unprepared for the fundamental SE needs of traceability, observability, and revision control.

On one extreme, powerful command-line interfaces like Claude Code and other CLI platforms grant developers immense control and flexibility. However, these platforms often result in ephemeral interactions. The rich conversational context (the back-and-forth dialogue of planning, clarification, and refinement between the human and the agent) is lost today, existing only in a terminal’s scroll-back buffer. The lack of systematic archival of the agent’s reasoning or the human’s guidance makes it nearly impossible to reconstruct the evolution of design decisions or reproduce specific outcomes, a shortcoming that we previously underscored in our SE 3.0 call for a new paradigm of conversational development [19].

On the other extreme, more integrated platforms like GitHub Copilot are much further ahead in addressing this by anchoring human-agent interactions to pull requests, thereby creating a persistent historical record. The agent’s suggestions and the resulting code changes are tracked. However, these systems treat the agent mentoring and the code as separate, unlinked artifacts. One can roll back a code change, but this does not roll back the state of the agent or the conversational thread that produced the code. The causal link between a specific piece of mentorship and its materialization in code is not explicitly maintained. This creates a fragmented history, in which the rationale for a change is decoupled from its implementation.

Moreover, no mainstream system today provides adequate observability into the agent’s internal state or a unified, interlinked revision control system for the combination of code, prompts, and conversational context. The artifacts of this new process are not being managed with the same rigor as traditional code. For agentic SE to mature from a craft into a true engineering discipline, we must develop new foundational building blocks that systematically support this new way of working, ensuring that the entire human-agent collaboration is observable, versionable, and trustworthy.

6.2 Embracing the Bitter Lesson in the Agentic SE Era

At first glance, our emphasis on structured processes (from BriefingScripts to structured orchestration via LoopScripts) might seem to run counter to the core message of Rich Sutton’s “**Bitter Lesson**” [42]. That lesson powerfully argues

that general methods that scale with computation and data ultimately triumph over approaches that rely on baking in specific human knowledge and processes. One might therefore think that our attempts to structure Agentic SE are a futile effort to impose human-centric designs on this new era.

However, the lesson’s power is most potent where data is abundant like at lower levels of abstraction or for common problems like building a web application. Its application becomes far more complex for novel tasks or in niche domains where training data is scarce. For these settings, relying solely on large-scale data is inefficient; a human is still needed to provide the overarching structure and connect the dots. This need arises from fundamental constraints inherent in both current and future AI: they lack embodied human experience necessary for physical-world verification, contextual judgment, and deep ethical reasoning [43]. SASE therefore does not merely encode human knowledge. It recognizes that designers must build human-AI partnerships around complementary roles. In this partnership, humans provide strategic and ethical guardrails with enough precision for AI agents to act. Even within that unique software system, there may be repetitive sub-problems where an agent can and should be granted full autonomy. We also note that in any system, a baseline of structured interactions, a Standard Operating Procedure (SOP), is essential for coordination and governance, especially in regulated settings.

Therefore, our vision does not reject the power of scale; it seeks to create the conditions for it to succeed reliably and reproducibly within a complex engineering context. Furthermore, our focus extends beyond mere process control to address the critical SE needs of traceability, robust human-agent communication [53], and fostering an agent-first mindset in how we structure codebases and design tools. This reinforces the idea that SE, especially in this new era, is as much about the journey (the collaboration, the evolution of intent, the trail of decisions) as it is about delivering the final outcome.

Ultimately, we believe software engineers must embrace the Bitter Lesson as a guiding principle. The core skill of the modern super software engineer is mastering the duality of control: strategically deciding when to impose a structured workflow (for novel, high-level tasks) and when to “let the agent loose” on a well-defined problem where it can leverage scaled learning. This is analogous to managing a team of brilliant experts; one must know when to provide a detailed work order and when to trust a team member with full autonomy.

6.3 Agentic Software Engineering, Not Just Agentic Coding: The Centrality of N-to-N Collaboration

A foundational premise of this paper is the distinction between agentic coding and agentic software engineering. Agentic coding, which characterizes the current state of most available tools, focuses primarily on the 1-to-1 interaction between a developer and an AI assistant to accelerate implementation tasks. Agentic coding is fundamentally an augmentation of a solo activity, aimed at boosting individual productivity.

Software Engineering (SE), by contrast, has always been a **team sport**. SE involves not only producing code but also managing complexity, coordinating across diverse roles, reconciling competing stakeholder needs, and ensuring the long-term sustainability of shared artifacts. These inherently collective challenges demand the acceleration of structured collaboration, not just individual acceleration.

Structured Agentic Software Engineering (SASE) is explicitly designed for this broader scope. It provides the artifacts, processes, and workbenches necessary to support **N-to-N collaboration**, where many humans and many agents interact as a coordinated team. This model manifests at several levels:

- **1-to-N Human-Agent Collaboration:** A single human orchestrates and mentors a fleet of agents, directing parallelized workstreams.

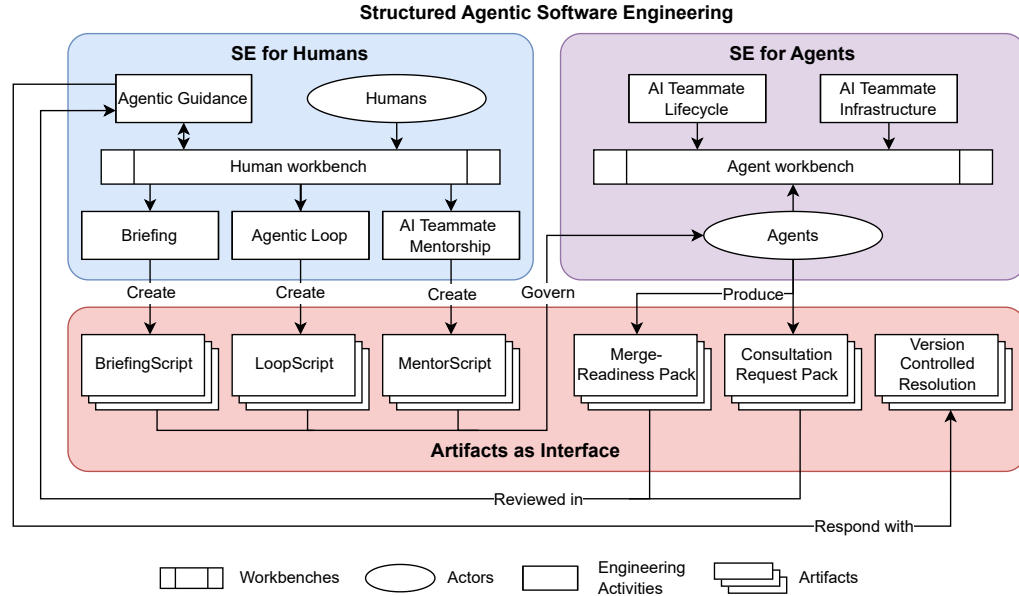


Fig. 3. The Structured Agentic Software Engineering (SASE) framework: dual domains for humans and agents, engineering activities, and artifacts.

- **1-to-N Agent-Human Collaboration:** An agent escalates a Consultation Request Pack (CRP) to the appropriate human specialist, enabling targeted, domain-specific feedback.
- **N-to-N Hybrid Collaboration:** Multiple humans collectively oversee and mentor a shared pool of agents, while agents collaborate with one another or even with specialized sub-agents.

For instance, an agent may route a database schema issue to the designated database architect, who provides feedback through the ACE. In more advanced settings, that “architect” role may itself be filled by another specialized agent. Without explicit, durable artifacts such as the CRP, these complex multi-actor workflows would be ephemeral, untraceable, and ultimately unmanageable. SASE ensures that these interactions leave behind structured, auditable records, transforming ad-hoc agentic coding into a disciplined engineering practice.

7 From Vision to Reality: A Research Roadmap for Structured Agentic SE and Its Implications

The research directions outlined below are not intended to be comprehensive; rather, they catalyze a shift in the community’s focus toward the novel challenges of engineering with and for intelligent agents. Fig. 3 outlines the relations between the tools, activities/processes, actors, and artifacts across the two dualities.

7.1 Briefing Engineering (BriefingEng)

Formalizing BriefingScript. Research is needed on languages and schemas that express goals, constraints, invariants, domain context, and acceptance criteria without forcing premature design decisions. The central challenge is balancing expressive power, learnability, and machine-checkable structure.

AI-Powered Authoring and Review. Briefing tools should help coaches detect ambiguity, surface missing context, generate edge cases, and check consistency across clauses. A key question is how such assistants can steer engineers toward property-based acceptance criteria rather than brittle examples.

Traceability and Multi-Solution Review. Trustworthy briefs require traceability from generated code and evidence back to the briefing clauses that motivated them. This need becomes sharper under N-version programming, where coaches must compare, combine, and justify choices across multiple agent-generated alternatives. Recent work on cognitive observability, such as Watson, offers an early foundation, but SASE requires observability across interacting artifacts, agents, and review decisions.

7.2 Agentic Loop Engineering (ALE)

Designing LoopScript. A declarative LoopScript language should capture task decomposition, parallel execution, review checkpoints, escalation rules, and evidence requirements. Research can build on business-process and DevOps automation, but SE-specific workflows need stronger links to code, tests, risks, and human review.

Human-in-the-Loop Control. Agents will explore unproductive paths, misread constraints, or need domain judgment. We need interaction mechanisms that let a coach pause a workflow, redirect a branch, add context, or stop low-value work without restarting the whole loop.

Evidence Packs and Feedback Signals. Merge-Readiness Packs require standards for sufficient evidence of correctness, security, quality, and rationale. Tool feedback also needs to become more informative for agents: a structured compiler diagnostic, test failure, or static-analysis finding can guide the search far better than an opaque failure signal.

7.3 AI Teammate Mentorship Engineering (ATME)

Developing MentorScript. MentorScript needs abstractions for rules that range from concrete style constraints to architectural principles. The language must be expressive enough for nuanced guidance, but simple enough for teams to review as ordinary engineering artifacts.

Quality Assurance for Mentorship Rules. If mentorship becomes code, it needs linting, testing, conflict detection, and regression checks. Researchers should study how to verify that new rules improve agent behavior without causing unintended changes elsewhere.

Learning and Explaining Team Norms. Agents may infer candidate MentorScript rules from repeated human feedback, but those rules should remain reviewable and auditable. Developers also need explanations that connect an agent’s decision to the specific rules considered, drawing on prompt interpretation and reasoning-observability techniques such as PromptExp [12] and Watson [38].

7.4 AI Teammate Lifecycle Engineering (ATLE)

Models for Persistent Agent Memory. Agents need mechanisms for retaining project history, decision logs, architectural rationale, and lessons from review. This includes both continual-learning methods [16, 41] and external memory structures such as graphs, vector stores, and decision records. Since unbounded history can overflow context windows or introduce irrelevant evidence, SE-specific memory compression must preserve semantic details in code, builds, tests, and dependencies [9, 13, 51, 55].

Proactive Maintenance. Persistent agents can scan for technical debt, documentation gaps, fragile tests, or refactoring opportunities during idle cycles. Research should study how to schedule such work, estimate its value, and present proposals without distracting teams from higher-priority development.

Economics of Agent-First Code. Agentic SE may change the cost model behind long-standing principles. For example, duplication may be easier for agents to update consistently, while strong type systems may become even more valuable because compiler feedback helps agents repair mistakes. These claims need empirical and economic models rather than anecdotes.

7.5 AI Teammate Infrastructure Engineering (ATIE)

The Post-IDE Human-Agent Interface. If agents perform much of the direct editing, the human-facing environment becomes a command center for specifying intent, comparing alternatives, routing consultations, and auditing evidence. Research should move toward interfaces for orchestration, review, and structured mentorship.

Distributed Compute Fabrics for Agents. Multi-agent SE needs runtime support for isolation, reproducibility, scheduling, and cost control. The declarative nature of LoopScript can expose workflow structure for optimization, connecting SASE to related work on SLA-aware and context-aware CodeLLM serving [45].

Agent-Native Toolchains. Today’s tools expose interfaces optimized for humans, so agents often fall back to brittle textual search and shell commands. Future toolchains should provide machine-readable protocols, structured diagnostics, semantic search, and self-describing operations that agents can use and improve over time.

7.6 The Human Differentiator: A Call to Reimagine SE Education

The SE field is defined by its four pillars: actors, processes, tools, and artifacts. While the SASE vision provides a structured approach for the latter three, we must not forget that the actor (the human engineer) remains the most critical factor. The 100x and 1,000x productivity gains emerging today are not the result of magical tools, but of skilled individuals who have mastered the art of working with agents. They demonstrate that even with today’s nascent technology, the human’s ability to specify intent and provide strategic oversight is the ultimate differentiator.

Even if the entire SASE framework were realized tomorrow, it would not automatically create a generation of 1,000x engineers. It would provide the scaffolding, but the ability to use that scaffolding effectively (to excel at Briefing Engineering, to design elegant LoopScripts, to codify insightful mentorship) will still separate the exceptional from the average. The human is not being automated away, but being elevated from a crafter of code to a conductor of agents.

This elevates a critical challenge that extends beyond research: we must fundamentally rethink SE education. Current curricula are largely designed to train students to *be* the agent (to write code, create tests, and handle low-level implementation). In the agentic era, we must instead train them to *manage* fleets of agents. This requires a profound pedagogical shift away from pure implementation and toward strategic skills: system-level thinking, architectural reasoning, rigorous specification, and the art of mentorship-as-code. Rather than simply adding a “prompt engineering” module to an existing course, this is a call for a deep and holistic reimagining of what it means to educate the next generation of software engineers. This challenge is further compounded by the concurrent shift in the nature of software itself towards “FMware” (FM-powered applications), a separate but equally transformative trend that also demands its own educational rethinking, though we do not address that here.

8 Related Efforts to Agentic Software Engineering

8.1 Iterative and Prompt-Driven Workflows

PDAR with Product Requirement Prompts (PRPs). The Plan-Do-Assess-Review loop formalizes a single task’s lifecycle: a human and AI plan, a dev-agent implements, an agent self-assesses, and a human reviews. PRPs act as the “minimum viable packet” capturing goals, justification, acceptance criteria, and curated context. Industry tools such as Amazon’s Kiro⁹ already demonstrate this “spec-driven development” pattern, typically structuring a PRP into five sections: (1) Goal & Why, setting the objective and business value; (2) What & Success Criteria, defining scope with verifiable conditions and invariants; (3) All Needed Context, curating relevant documentation and known pitfalls without overloading the agent’s context; (4) Implementation Blueprint, providing strategic guidance and constraints rather than a low-level plan; and (5) Validation Loop, codifying the acceptance testing strategy. This aligns with SASE’s insistence on structured, testable intent. However, PDAR is scoped to one-off execution; it does not by itself establish durable mentorship, agent lifecycle learning, or cross-task traceability that SASE treats as first-class.

Agent Skills and Plugins. Superpowers¹⁰ encapsulate best-practice prompts into predefined skills/plugins for agents to perform tasks like brainstorming, reviewing code, and running debugging workflows. These skills raise consistency and convenience for solo developers but stop short of a team-level methodology. They neither codify mentorship as versioned rules nor address observability or merge-readiness evidence as explicit deliverables.

Meta-prompt files (AGENT.md, CLAUDE.md). Practitioners use project-level configuration files (e.g., CLAUDE.md, .clinerules, AGENT.md) to load “institutional knowledge” before every agent task, codifying style guides, architectural constraints, and lessons learned into a continuously improving “employee handbook” for AI teammates. This grassroots practice highlights an open question: the community has no consensus on what such files should contain or the appropriate level of detail, underscoring that future work involves not only defining languages like MentorScript but also discovering best practices for their effective use.

8.2 Multi-Agent and Agile-Inspired Frameworks

BMAD (Breakthrough Method for Agile AI-Driven Development). BMAD¹¹ organizes agents into agile roles (e.g., Product Owner, Architect, Developer, Tester). Up-front agentic planning yields PRDs and designs; a Scrum-like shard step creates “story files” with focused context; specialized agents execute in parallel. BMAD’s strengths (role specialization, task sharding, and high parallelism) map well to SASE’s N-version programming and orchestration. SASE goes further by (i) converting review feedback into persistent MentorScript rules (mentorship-as-code), and (ii) specifying the environments and disciplines (ACE for human coaching and orchestration, AEE for agent execution, and ATLE/ATIE for memory, lifecycle, and agent-native tooling).

8.3 Future of Software Engineering

Vision and Roadmap. Recent papers describe a shift from code-completion tools toward socio-technical systems in which developers specify intent, coordinate agents, evaluate evidence, and remain accountable for outcomes. Ahmed et al. [2] survey the AI for SE landscape and identify trust in AI-generated code and human-AI collaboration as key open challenges. Gao et al. [14] emphasize hallucinations, limited contextual understanding, and shortcomings in

⁹<https://kiro.dev/blog/introducing-kiro/>

¹⁰<https://github.com/obra/Superpowers>

¹¹<https://github.com/bmad-code-org/BMAD-METHOD>

current evaluation methods. Terragni et al. [44] and Qiu et al. [36] extend this trajectory to full-lifecycle AI-driven development and the changing daily routine of developers. He et al. [21] focus on role-specialized multi-agent systems, while Abrahão et al. [1] emphasize developer experience, cognitive load, and human accountability.

8.4 How SASE Aligns and Differentiates

Alignment. SASE adopts the best of these efforts, such as PRP-style briefs for intent formalization, PDAR-style iterative loops, and BMAD-like multi-agent parallelism.

Differentiation. SASE elevates these pieces into a holistic engineering methodology with four distinguishing features:

- (1) **Mentorship-as-Code (ATME).** Review guidance becomes a version-controlled, testable MentorScript, enabling cumulative, auditable improvement across tasks and teams.
- (2) **Dual Workbenches.** The Agent Command Environment (ACE) optimizes human cognition for specification, orchestration, and evidence-based review; the Agent Execution Environment (AEE) optimizes for agent strengths (e.g., massive parallelism).
- (3) **Merge-Readiness as the Target Artifact.** The loop’s output is a Merge-Readiness Pack, which is a progressive-disclosure bundle proving functional completeness, sound verification, SE hygiene, rationale, and full auditability.
- (4) **Consultability as a First-Class Artifact.** SASE introduces the Consultation Request Pack as a structured artifact for agent-initiated human consultation. This elevates humans to callable experts and enables traceable cross-role handovers, shifting the paradigm from solo agentic coding to team-based Agentic Software Engineering.
- (5) **Lifecycle & Infrastructure (ATLE & ATIE).** Agents become persistent teammates with memory, observability, and secure, hermetic execution, shifting from stateless contractors to evolving collaborators.

9 Conclusion

Agentic Software Engineering (SE 3.0) demands more than incremental adjustments to existing SE practices. In this paper, we presented Structured Agentic Software Engineering (SASE), a conceptual framework for making agentic SE more structured, predictable, and trustworthy. Our core contribution is a duality between SE for Humans and SE for Agents, which reimagines the field’s actors, processes, tools, and artifacts around human-agent collaboration.

SASE operationalizes this duality through dedicated environments (ACE and AEE) and version-controlled artifacts (BriefingScript, LoopScript, MentorScript, CRP, and MRP). These mechanisms shift the human role from direct implementer to strategic “Agent Coach and Orchestrator,” with important implications for SE education.

We offer SASE not as a final solution, but as a scaffold for community discussion and refinement. By building disciplined foundations for human-agent collaboration, the SE community can move beyond impressive but brittle demonstrations. The future of SE will be defined not by agent speed alone, but by our ability to mentor, orchestrate, and trust agents as engineering partners.

References

- [1] Silvia Abrahão, John Grundy, Mauro Pezzè, Margaret-Anne Storey, and Damian A. Tamburri. 2025. Software Engineering by and for Humans in an AI Era. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 129 (May 2025), 46 pages. doi:10.1145/3715111
- [2] Iftekhar Ahmed, Aldeida Aleti, Haipeng Cai, Alexander Chatzigeorgiou, Pinjia He, Xing Hu, Mauro Pezzè, Denys Poshyvanyk, and Xin Xia. 2025. Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 119 (May 2025), 27 pages. doi:10.1145/3719006
- [3] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. *SWE-Bench+: Enhanced Coding Benchmark for LLMs*. arXiv:2410.06992 [cs.SE]

- [4] Sayed Mahbub Hasan Amiri, Md. Mainul Islam, Mohammad Shakhawat Hossen, Sayed Majhab Hasan Amiri, Mohammad Shawkat Ali Mamun, Sk. Humaun Kabir, and Naznin Akter. 2025. *Hear Your Code Fail, Voice-Assisted Debugging for Python*. arXiv:2507.15007 [cs.PL]
- [5] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. *SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents*. arXiv:2505.20411 [cs.SE]
- [6] S.S. Brilliant, J.C. Knight, and N.G. Leveson. 1989. The consistent comparison problem in N-version software. *IEEE Transactions on Software Engineering* 15, 11 (1989), 1481–1485. doi:10.1109/32.41339
- [7] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884 [cs.SE]
- [8] Liming Chen and A. Avizienis. 1995. N-version programming: A fault-tolerance approach to reliability of software operation. In *Twenty-Fifth International Symposium on Fault-Tolerant Computing, 1995. 'Highlights from Twenty-Five Years'*. IEEE Computer Society, Los Alamitos, CA, USA, 113. doi:10.1109/FTCSH.1995.532621
- [9] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting Language Models to Compress Contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, 3829–3846. doi:10.18653/V1/2023.EMNLP-MAIN.232
- [10] Filipe Roseiro Cogo, Gopi Krishnan Rajbahadur, Dayi Lin, Keheliya Gallaba, Benjamin Rombaut, Gustavo Oliva, Jiahuei (Justina) Lin, Kirill Vasilevski, and Ahmed E. Hassan. 2025. A Tutorial on Software Engineering for Fmware. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion 2025, Clarion Hotel Trondheim, Trondheim, Norway, June 23-28, 2025*, Leonardo Montecchi, Jingyue Li, Denys Poshyvanyk, and Dongmei Zhang (Eds.). ACM, 1231–1233. doi:10.1145/3696630.3728621
- [11] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941 doi:10.48550/ARXIV.2509.16941
- [12] Ximing Dong, Shaowei Wang, Dayi Lin, Gopi Krishnan Rajbahadur, and Ahmed E. Hassan. 2025. Promptexp: Multi-Granularity Prompt Explanation of Large Language Models. (2025), 1–10. doi:10.1109/AIWARE69974.2025.00027
- [13] Weizhi Fei, Xueyan Niu, Pingyi Zhou, Lu Hou, Bo Bai, Lei Deng, and Wei Han. 2024. Extending Context Window of Large Language Models via Semantic Compression. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024 (Findings of ACL, Vol. ACL 2024)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 5169–5181. doi:10.18653/V1/2024.FINDINGS-ACL.306
- [14] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The Current Challenges of Software Engineering in the Era of Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 127 (May 2025), 30 pages. doi:10.1145/3712005
- [15] Kajsa Gullberg, Victoria Johansson, and Roger Johansson. 2024. In Scriptura Veritas? Exploring Measures for Identifying Increased Cognitive Load in Speaking and Writing. *Languages* 9, 3 (2024). doi:10.3390/languages9030085
- [16] Haiyang Guo, Fanhu Zeng, Fei Zhu, Jiayi Wang, Xukai Wang, Jingang Zhou, Hongbo Zhao, Wenzhuo Liu, Shijie Ma, Da-Han Wang, Xu-Yao Zhang, and Cheng-Lin Liu. 2025. A Comprehensive Survey on Continual Learning in Generative Models. *CoRR* abs/2506.13045 (2025). arXiv:2506.13045 doi:10.48550/ARXIV.2506.13045
- [17] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2025. *Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers*. arXiv:2506.13538 [cs.SE]
- [18] Ahmed E. Hassan. 2008. The road ahead for Mining Software Repositories. In *2008 Frontiers of Software Maintenance*. 48–57. doi:10.1109/FOSM.2008.4659248
- [19] Ahmed E. Hassan, Gustavo A. Oliva, Dayi Lin, Boyuan Chen, and Zhen Ming Jiang. 2024. *Towards AI-Native Software Engineering (SE 3.0): A Vision and a Challenge Roadmap*. arXiv:2410.06107 [cs.SE]
- [20] Ahmed E. Hassan and Tao Xie. 2010. Mining software engineering data. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 2 (Cape Town, South Africa) (ICSE '10)*. Association for Computing Machinery, New York, NY, USA, 503–504. doi:10.1145/1810295.1810451
- [21] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. doi:10.1145/3712003
- [22] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. arXiv:2511.04824 [cs.SE]
- [23] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. *CoRR* abs/2409.12186 (2024). arXiv:2409.12186 doi:10.48550/ARXIV.2409.12186
- [24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

- [25] Hao Li, Cor-Paul Bezemer, and Ahmed E. Hassan. 2025. Software Engineering and Foundation Models: Insights from Industry Blogs Using a Jury of Foundation Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 307–318. doi:10.1109/ICSE-SEIP66354.2025.00033
- [26] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. *The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering*. arXiv:2507.15003 [cs.SE]
- [27] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2026. AIDev: Studying AI Coding Agents on GitHub. *CoRR* abs/2602.09185 (2026). arXiv:2602.09185 doi:10.48550/ARXIV.2602.09185
- [28] Daniel Liew, Daniel Schemmel, Cristian Cadar, Alastair F. Donaldson, Rafael Zahl, and Klaus Wehrle. 2017. Floating-point symbolic execution: A case study in N-version programming. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 601–612. doi:10.1109/ASE.2017.8115670
- [29] Zexiong Ma, Chao Peng, Pengfei Gao, Xiangxin Meng, Yanzhen Zou, and Bing Xie. 2025. SoRFT: Issue Resolving with Subtask-oriented Reinforced Fine-Tuning. (2025), 11427–11441. doi:10.18653/V1/2025.ACL-LONG.559
- [30] Audris Mockus, Roy T. Fielding, and James D. Herbsleb. 2002. Two case studies of open source software development: Apache and Mozilla. *ACM Trans. Softw. Eng. Methodol.* 11, 3 (July 2002), 309–346. doi:10.1145/567793.567795
- [31] OpenAI. 2024. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>.
- [32] OpenAI. 2026. Why We No Longer Evaluate SWE-bench Verified. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>.
- [33] Youssef Esseddiq Ouattiti, Mohammed Sayagh, Hao Li, and Ahmed E. Hassan. 2026. Do AI Coding Agents Log Like Humans? An Empirical Study. arXiv:2604.09409 [cs.SE]
- [34] R. Parasuraman, T.B. Sheridan, and C.D. Wickens. 2000. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30, 3 (2000), 286–297. doi:10.1109/3468.844354
- [35] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable Code Generation from Pre-trained Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net.
- [36] Ketai Qiu, Niccolò Puccinelli, Matteo Ciniselli, and Luca Di Grazia. 2025. From Today’s Code to Tomorrow’s Symphony: The AI Transformation of Developer’s Routine by 2030. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 121 (May 2025), 17 pages. doi:10.1145/3709353
- [37] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *International Conference on Machine Learning, ICML 2023, 23–29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 28492–28518.
- [38] Benjamin Rombaut, Sogol Masoumzadeh, Kirill Vasilevski, Dayi Lin, and Ahmed E. Hassan. 2025. *Watson: A Cognitive Observability Framework for the Reasoning of LLM-Powered Agents*. arXiv:2411.03455 [cs.AI]
- [39] Javier Ron, Diogo Gaspar, Javier Cabrera-Arteaga, Benoit Baudry, and Martin Monperrus. 2025. Galápagos: Automated N-Version Programming with LLMs. *ACM Transactions on Software Engineering and Methodology* (Dec. 2025). doi:10.1145/3785363
- [40] Alexandru Constantin Serban, Erik Poll, and Joost Visser. 2018. A Standard Driven Software Architecture for Fully Autonomous Vehicles. (2018), 120–127. doi:10.1109/ICSA-C.2018.00040
- [41] Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. 2026. Continual Learning of Large Language Models: A Comprehensive Survey. *ACM Comput. Surv.* 58, 5 (2026), 120:1–120:42. doi:10.1145/3735633
- [42] Richard Sutton. 2019. The bitter lesson. *Incomplete Ideas (blog)* 13, 1 (2019), 38.
- [43] AI Swiss. 2025. *For a Swiss education transformed by AI*. White paper.
- [44] Valerio Terragni, Annie Vella, Partha Roop, and Kelly Blincoe. 2025. The Future of AI-Driven Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 120 (May 2025), 20 pages. doi:10.1145/3715003
- [45] Kishanthan Thangarajah, Boyuan Chen, Shi Chang, and Ahmed E. Hassan. 2025. *Context-Aware CodeLLM Eviction for AI-assisted Coding*. arXiv:2506.18796 [cs.SE]
- [46] Haoxin Tu, Huan Zhao, Yahui Song, Mehtab Zafar, Ruijie Meng, and Abhik Roychoudhury. 2026. Agentic Verification of Software Systems. arXiv:2511.17330 [cs.SE] doi:10.48550/arXiv.2511.17330
- [47] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net.
- [48] You Wang, Michael Pradel, and Zhongxin Liu. 2025. *Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study*. arXiv:2503.15223 [cs.SE]
- [49] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2026. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *ACM Trans. Softw. Eng. Methodol.* (March 2026). doi:10.1145/3798166
- [50] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. *CoRR* abs/2502.18449 (2025). arXiv:2502.18449 doi:10.48550/ARXIV.2502.18449

- [51] David Wingate, Mohammad Shoenybi, and Taylor Sorensen. 2022. Prompt Compression and Contrastive Conditioning for Controllability and Toxicity Reduction in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022 (Findings of ACL, Vol. EMNLP 2022)*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, 5621–5634. [doi:10.18653/V1/2022.FINDINGS-EMNLP.412](https://doi.org/10.18653/V1/2022.FINDINGS-EMNLP.412)
- [52] T. Winters, T. Manshreck, and H. Wright. 2020. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly Media.
- [53] Jie JW Wu and Fatemeh H. Fard. 2025. HumanEvalComm: Benchmarking the Communication Competence of Code Generation for LLMs and LLM Agents. *ACM Trans. Softw. Eng. Methodol.* 34, 7, Article 189 (Aug. 2025), 42 pages. [doi:10.1145/3715109](https://doi.org/10.1145/3715109)
- [54] Wendong Xu, Jing Xiong, Chenyang Zhao, Qiujiang Chen, Haoran Wang, Hui Shen, Zhongwei Wan, Jianbo Dai, Taiqiang Wu, He Xiao, Chaofan Tao, Z. Morley Mao, Ying Sheng, Zhijiang Guo, Hongxia Yang, Bei Yu, Lingpeng Kong, Quanquan Gu, and Ngai Wong. 2025. *SwingArena: Competitive Programming Arena for Long-context GitHub Issue Solving*. arXiv:2505.23932 [cs.CL]
- [55] Zhaozhuo Xu, Zirui Liu, Beidi Chen, Shaochen (Henry) Zhong, Yuxin Tang, Jue Wang, Kaixiong Zhou, Xia Hu, and Anshumali Shrivastava. 2024. Soft Prompt Recovers Compressed LLMs, Transferably. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024 (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR / OpenReview.net, 55186–55203.
- [56] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research, Vol. 267)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR / OpenReview.net.

A Examples of SASE Artifacts

To make the SASE framework more concrete, this appendix provides illustrative examples of three key artifacts that span the complete task lifecycle: (1) a BriefingScript that initiates work, (2) a Consultation Request Pack (CRP) generated during execution when human guidance is required, and (3) a Merge-Readiness Pack (MRP) submitted upon task completion. Together, these examples illustrate the structured dialogue between the Agent Command Environment (ACE) and the Agent Execution Environment (AEE).

A.1 BriefingScript Example

Listing 1 shows a BriefingScript for implementing rate limiting on a REST API. The example follows the five-section structure described in Section 5: (1) Goal & Why, (2) What & Success Criteria, (3) All Needed Context, (4) Implementation Blueprint, and (5) Validation Loop.

Listing 1. A BriefingScript for implementing API rate limiting.

```
# BriefingScript: Implement Rate Limiting for REST API
briefing_id: BS-2025-API-012
version: 1.2

# Section 1: Goal & Why
goal:
  objective: "Add rate limiting to prevent API abuse."
  business_value: "Protects infrastructure from DDoS,
                  reduces costs, improves reliability."

# Section 2: What & Success Criteria
success_criteria:
  definition_of_done:
    - "Rate limiter enforces 100 req/min per API key."
    - "Exceeded requests return HTTP 429 with Retry-After."
  invariants:
    - "Rate limiting must be idempotent across instances."
  preconditions:
    - "Valid API key exists in request header."

# Section 3: All Needed Context
context:
  relevant_files:
    - "src/middleware/auth.ts"
    - "src/config/rate-limits.yaml"
  documentation:
    - "docs/architecture/api-gateway.md"
  known_gotchas:
    - "Redis cluster has ~50ms replication lag."
    - "Legacy /v1/* endpoints must remain unlimited."

# Section 4: Implementation Blueprint
blueprint:
  recommended_approach: "Use sliding window with Redis."
  constraints:
    - "Do NOT modify auth middleware directly."
    - "Use existing Redis pool from src/db/redis.ts."
  patterns_to_use:
    - "Strategy pattern for swappable algorithms."

# Section 5: Validation Loop
validation:
  levels:
    - type: "unit"
      description: "Test sliding window with mock Redis."
    - type: "integration"
      description: "Verify 429 responses under load."
    - type: "acceptance"
      description: "Confirm Retry-After per RFC 7231."
```

A.2 Consultation Request Pack (CRP) Example

Listing 2 illustrates a Consultation Request Pack generated by an agent when it encounters an ambiguity that requires human judgment. In this scenario, the agent working on the BriefingScript from Listing 1 identifies a conflict between the specified Redis backend and the known gotcha about replication lag for high-stakes endpoints. The CRP structures the decision point with options, trade-offs, and a recommendation, enabling efficient human intervention.

Listing 2. A Consultation Request Pack (CRP) for an architectural decision.

```
# Consultation Request Pack: Architectural Decision
crp_id: CRP-2025-API-012-001
source_agent: "developer-agent-03"
triggered_by: "BriefingScript BS-2025-API-012"
timestamp: "2025-06-15T14:32:00Z"

# The Decision Point
issue:
  summary: "Conflicting requirements for rate limit backend."
  details: |
    The BriefingScript specifies Redis, but known_gotchas
    warn about replication lag. For /v2/payments (high-stakes),
    even 50ms inconsistency could allow burst abuse.

# Structured Options for Human Decider
options:
  - id: A
    label: "Redis with local cache fallback"
    pros: ["Matches blueprint", "Simple implementation"]
    cons: ["Inconsistency window during failover"]
    estimated_effort: "2 hours"

  - id: B
    label: "Redis with synchronous replication"
    pros: ["Strong consistency for /v2/payments"]
    cons: ["Higher latency (~100ms)", "DevOps coordination"]
    estimated_effort: "4 hours"

# Agent's Recommendation
recommendation:
  preferred_option: "B"
  reasoning: |
    For financial transactions, consistency outweighs
    latency cost. 100ms overhead is acceptable.

# Request for Human
requested_decision: "Confirm Option A or B, or provide
                    alternative guidance."
escalation_target: "Tech Lead or Architect role"
```

A.3 Merge-Readiness Pack (MRP) Example

Listing 3 shows a Merge-Readiness Pack submitted by the agent upon completing the rate limiting implementation from Listing 1. Following the human's approval of Option B in the CRP (Listing 2), the agent implemented synchronous replication for the /v2/payments endpoint. The MRP bundles evidence demonstrating functional completeness, sound verification, SE hygiene, clear rationale, and full auditability, which are the five criteria for merge-readiness defined in Section 5.

Listing 3. A Merge-Readiness Pack (MRP) for the completed rate limiting feature.

```

# Merge-Readiness Pack: Rate Limiting Implementation
mrp_id: MRP-2025-API-012
source_briefing: "BS-2025-API-012 (v1.2)"
agent: "developer-agent-03"
submitted: "2025-06-15T18:45:00Z"

# 1. Functional Completeness
functional_completeness:
  criteria_met:
    - criterion: "100 req/min per API key"
      status: PASS
      evidence: "integration_test_rate_limit.log"
    - criterion: "HTTP 429 with Retry-After header"
      status: PASS
      evidence: "test_429_response.log"
  notes: "Option B implemented per CRP-2025-API-012-001"
  resolution: "/v2/payments uses sync replication."

# 2. Sound Verification
verification:
  unit_tests: {total: 24, passed: 24, coverage: "92%"}
  integration_tests: {total: 8, passed: 8}
  load_test:
    tool: "k6"
    result: "429 returned at 101st request within 60s"
  acceptance_test:
    description: "Retry-After header format per RFC 7231"
    status: PASS

# 3. SE Hygiene
hygiene:
  linting: {warnings: 0, errors: 0}
  complexity:
    cyclomatic_max: 6
    threshold: 10
    status: PASS
  duplication: "No new duplicated blocks detected"

# 4. Rationale & Communication
rationale:
  approach: |
    Implemented sliding window algorithm with Redis backend.
    Used Strategy pattern for algorithm flexibility.
    Added sync replication for /v2/payments per architect
    guidance (see CRP-2025-API-012-001 resolution).
  trade_offs: |
    Chose consistency over latency for payment endpoints.
    ~100ms overhead acceptable for financial transactions.

# 5. Auditability
auditability:
  briefing_hash: "sha256:a1b2c3..."
  mentorscript_version: "v2.1.0"
  crp_resolutions: ["CRP-2025-API-012-001"]
  trajectory_id: "trace-2025-06-15-agent03-api012"
  artifacts:
    - "src/middleware/rate-limiter.ts"
    - "src/middleware/rate-limiter.test.ts"
    - "docs/rate-limiting.md"

```